04 Graphs

2-SAT

O(n + m)

SCC, implication graph, satisfiability

```
struct TwoSat {
    int n;

    vector<vector<int>> g, rg;
    vector<int> comp, order;
    vector<char> used;
    vector<bool> assignment;

    TwoSat(int n)
        : n(n),
          g(2 * n),
          rg(2 * n),
          comp(2 * n, -1),
          order(),
          used(2 * n, false),
          assignment(n, false) {}

    int node(int x, bool neg) {
        return 2 * x + neg;
    }

    void add_clause(
        int a, bool na,
        int b, bool nb
    ) {
        int A = node(a, na);
        int B = node(b, nb);
        g[A ^ 1].push_back(B);
        g[B ^ 1].push_back(A);
        rg[B].push_back(A ^ 1);
        rg[A].push_back(B ^ 1);
    }

    void dfs1(int v) {
        used[v] = true;
        for (int u : g[v]) {
            if (!used[u])
                dfs1(u);
        }
        order.push_back(v);
    }

    void dfs2(int v, int c) {
        comp[v] = c;
        for (int u : rg[v]) {
            if (comp[u] == -1)
                dfs2(u, c);
        }
    }

    bool solve() {
        fill(used.begin(), used.end(), false);
        order.clear();
        for (int v = 0; v < 2 * n; v++) {
            if (!used[v])
                dfs1(v);
        }
        fill(comp.begin(), comp.end(), -1);
        int components = 0;
        for (int i = 2 * n - 1; i >= 0; i--) {
            int v = order[i];
            if (comp[v] == -1)
                dfs2(v, components++);
        }
        for (int i = 0; i < n; i++) {
            if (comp[2 * i] == comp[2 * i + 1])
                return false;
            assignment[i] = comp[2 * i] > comp[2 * i + 1];
        }
        return true;
    }
};
```

Bellman-Ford Algorithm

O(VE)

shortest path, negative weights

```
/*
@title: Bellman-Ford Algorithm
@category: Graphs
@complexity: O(VE)
@tags: shortest path, negative weights
*/

struct Edge {
    int a, b, cost;
};

int n;
vector<Edge> edges;
const int INF = 1000000000;

void solve() {
    vector<int> d(n, 0);
    vector<int> p(n, -1);
    int x;

    for (int i = 0; i < n; ++i) {
        x = -1;
        for (Edge e : edges) {
            if (d[e.a] + e.cost < d[e.b]) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
        }
    }

    if (x == -1) {
        cout << "No negative cycle found.";
    } else {
        for (int i = 0; i < n; ++i)
            x = p[x];

        vector<int> cycle;
        for (int v = x;; v = p[v]) {
            cycle.push_back(v);
            if (v == x && cycle.size() > 1)
                break;
        }
        reverse(cycle.begin(), cycle.end());

        cout << "Negative cycle: ";
        for (int v : cycle)
            cout << v << ' ';
        cout << endl;
    }
}
```

Eulerian Path

O(V^2 + E)

graph traversal, Hierholzer's algorithm

```
/*
@title: Eulerian Path
@category: Graphs
@complexity: O(V^2 + E)
@tags: graph traversal, Hierholzer's algorithm
@input: n, then n x n adjacency matrix (undirected multigraph, NO
     self-loops)
@output: sequence of vertices of an Eulerian path/circuit, or -1 if
      none exists
*/

int main() {
    int n;
    cin >> n;
    vector<vector<int>> g(n, vector<int>(n));
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> g[i][j];

    // Self-loops never change whether an Eulerian path/circuit
     exists (they
    // are entered and exited at the same vertex), but this matrix-
     based
    // traversal cannot consume them correctly (it decrements the
     same cell
    // twice per step and hangs forever). Strip them before running
     .
    // If your problem needs self-loops printed, remember how many
     each
    // vertex had and splice them into the output wherever that
     vertex
    // appears in the final trail.
    for (int i = 0; i < n; ++i)
        g[i][i] = 0;

    vector<int> deg(n);
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            deg[i] += g[i][j];

    int first = 0;
    while (first < n && !deg[first])
        ++first;
    if (first == n) {
        cout << -1 << "\n"; // no edges at all
        return 0;
    }

    int v1 = -1, v2 = -1;
    bool bad = false;
    for (int i = 0; i < n; ++i) {
        if (deg[i] & 1) {
            if (v1 == -1) v1 = i;
            else if (v2 == -1) v2 = i;
            else bad = true;
        }
    }

    if (v1 != -1)
        ++g[v1][v2], ++g[v2][v1];

    stack<int> st;
    st.push(first);
    vector<int> res;
    while (!st.empty()) {
```

```
        int v = st.top();
        int i;
        for (i = 0; i < n; ++i)
            if (g[v][i]) break;
        if (i == n) {
            res.push_back(v);
            st.pop();
        } else {
            --g[v][i];
            --g[i][v];
            st.push(i);
        }
    }

    if (v1 != -1) {
        for (size_t i = 0; i + 1 < res.size(); ++i) {
            if ((res[i] == v1 && res[i + 1] == v2) ||
                (res[i] == v2 && res[i + 1] == v1)) {
                vector<int> res2;
                for (size_t j = i + 1; j < res.size(); ++j)
                    res2.push_back(res[j]);
                for (size_t j = 1; j <= i; ++j)
                    res2.push_back(res[j]);
                res = res2;
                break;
            }
        }
    }

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (g[i][j]) bad = true; // leftover edges => graph was
      disconnected

    if (bad) {
        cout << -1 << "\n";
    } else {
        for (int x : res) cout << x << " ";
        cout << "\n";
    }
}
```

07 Dp

Divide & Conquer DP

O(m n log n)

DP optimization, monotone opt

```
/*
    DP of the form:
        dp[i][j] = min_{0 <= k <= j} dp[i-1][k-1] + C(k,j)
    where opt[i][j] is monotone:
        opt[i][j] <= opt[i][j+1]
    Sufficient condition:
        C(a,c) + C(b,d) <= C(a,d) + C(b,c)
        for a <= b <= c <= d.
    IMPORTANT:
        If exactly i groups are required, the valid range
        of k usually needs an additional lower bound.
*/

using ll = long long;

int n, m;
vector<ll> dp_before, dp_cur;

ll C(int k, int j);

void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;
    int mid = (l + r) >> 1;
    pair<ll, int> best = {LLONG_MAX,-1};
    for (int k = optl;k <= min(mid, optr);k++) {
        ll val =
            (k ? dp_before[k - 1] : 0)
            + C(k, mid);
        if (val < best.first)
            best = {val, k};
    }
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}

ll solve() {
    dp_before.assign(n, 0);
    dp_cur.assign(n, 0);
    for (int j = 0; j < n; j++)
        dp_before[j] = C(0, j);
    for (int i = 1; i < m; i++) {
        compute(0, n - 1, 0, n - 1);
        dp_before = dp_cur;
    }
    return dp_before[n - 1];
}
```

Knuth Optimization

O(n^2)

interval DP, DP optimization

```
/*
    dp[l][r] = minimum cost for interval [l,r).
    Transition:
        dp[l][r] = min(dp[l][k] + dp[k][r] + C(l,r))   for l < k <
     r
    Knuth condition:
        opt[l][r-1] <= opt[l][r] <= opt[l+1][r]
    Base:
        dp[i][i] = 0, dp[i][i+1] = 0 (single element, no split
     needed)
    Requires C to satisfy the quadrangle inequality + monotonicity
    (true e.g. for C(l,r) = prefix[r]-prefix[l] with non-negative
     weights).
*/

using ll = long long;
vector<vector<ll>> knuth(int n, const vector<ll>& prefix) {
    const ll INF = (1LL << 62);
    vector<vector<ll>> dp(n + 1, vector<ll>(n + 1, 0));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1, 0));

    for (int i = 0; i <= n; i++)
        opt[i][i] = i;
    for (int i = 0; i < n; i++)
        opt[i][i + 1] = i;   // seed the length-1 boundary (was
     left at 0 before!)

    auto C = [&](int l, int r) -> ll {
        return prefix[r] - prefix[l];
    };

    for (int len = 2; len <= n; len++) {
        for (int l = 0; l + len <= n; l++) {
            int r = l + len;
            dp[l][r] = INF;
            int L = max(opt[l][r - 1], l + 1);
            int R = min(opt[l + 1][r], r - 1);
            for (int k = L; k <= R; k++) {
                ll val = dp[l][k] + dp[k][r] + C(l, r);
                if (val < dp[l][r]) {
                    dp[l][r] = val;
                    opt[l][r] = k;
                }
            }
        }
    }

    return dp;
}
```

02 Math

Linear Diophantine Equation

O(log min(|a|,|b|))

extended gcd, ax+by=c, integer solutions

```
/*
    Solve:
        a*x + b*y = c
    All solutions:
        x = x0 + k*(b/g)
        y = y0 - k*(a/g)
    Returns false if no solution exists.
    Assumes a and b are not both zero.
*/

using ll = long long;

ll extgcd(ll a, ll b, ll& x, ll& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    ll x1, y1;
    ll g = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

ll mygcd(ll a, ll b) {
    while (b) { a %= b; swap(a, b); }
    return a;
}

// Returns g = gcd(a,b), any solution x0,y0 to a*x0+b*y0=c.
bool find_any_solution(ll a, ll b, ll c, ll& x0, ll& y0, ll& g) {
    if (a == 0 && b == 0)
        return false;
    g = mygcd(llabs(a), llabs(b));
    if (c % g != 0)
        return false;
    ll x, y;
    extgcd(llabs(a), llabs(b), x, y);
    x0 = x * (c / g);
    y0 = y * (c / g);
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}

ll floor_div(ll a, ll b) {
    ll q = a / b, r = a % b;
    if (r != 0 && ((r > 0) != (b > 0))) --q;
    return q;
}

ll ceil_div(ll a, ll b) { return -floor_div(-a, b); }

// Finds the range of x-values (lx..rx) among all solutions of a*x+
     b*y=c
// with x in [minx,maxx] and y in [miny,maxy].
// Number of such solutions = (rx-lx)/abs(b/g) + 1.
```

```
// NOTE: correctly handles negative/zero a or b (unlike the naive
// cp-algorithms-style version, which silently breaks on negative
     coeffs).
bool find_solution_range(
    ll a, ll b, ll c,
    ll minx, ll maxx,
    ll miny, ll maxy,
    ll& lx, ll& rx
) {
    ll x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return false;
    a /= g;
    b /= g;

    ll klo = LLONG_MIN / 4;
    ll khi = LLONG_MAX / 4;

    // minx <= x + k*b <= maxx
    if (b > 0) {
        klo = max(klo, ceil_div(minx - x, b));
        khi = min(khi, floor_div(maxx - x, b));
    } else if (b < 0) {
        klo = max(klo, ceil_div(maxx - x, b));
        khi = min(khi, floor_div(minx - x, b));
    } else if (x < minx || x > maxx) {
        return false; // b == 0 (a != 0): x is fixed, must lie in [
     minx,maxx]
    }

    // miny <= y - k*a <= maxy
    if (a > 0) {
        klo = max(klo, ceil_div(y - maxy, a));
        khi = min(khi, floor_div(y - miny, a));
    } else if (a < 0) {
        klo = max(klo, ceil_div(y - miny, a));
        khi = min(khi, floor_div(y - maxy, a));
    } else if (y < miny || y > maxy) {
        return false; // a == 0 (b != 0): y is fixed, must lie in [
     miny,maxy]
    }

    if (klo > khi)
        return false;

    lx = x + klo * b;
    rx = x + khi * b;
    if (lx > rx) swap(lx, rx);
    return true;
}
```

Miller-Rabin Primality Test

O(log^3 n) worst-case, 64-bit

primality, modular exponentiation

```
using ull = unsigned long long;
using u128 = __uint128_t;

ull mod_pow(ull a, ull e, ull mod) {
    ull r = 1;
    while (e) {
        if (e & 1)
            r = (u128)r * a % mod;
        a = (u128)a * a % mod;
        e >>= 1;
    }
    return r;
}

bool isPrime(ull n) {
    if (n < 2)
        return false;
for (ull p : {
        2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
        17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL
    }) {
        if (n % p == 0)
            return n == p;
    }
    ull d = n - 1;
    int s = 0;
    while ((d & 1) == 0) {
        d >>= 1;
        ++s;
    }
    for (ull a : {
        2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
        17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL
    }) {
        if (a >= n)
            continue;
        ull x = mod_pow(a, d, n);
        if (x == 1 || x == n - 1)
            continue;
        bool witness = true;
        for (int r = 1; r < s; ++r) {
            x = (u128)x * x % n;
            if (x == n - 1) {
                witness = false;
                break;
            }
        }
        if (witness)
            return false;
    }
    return true;
}
```

FFT / NTT Polynomial Multiplication

O(n log n)

convolution, polynomial multiplication, FFT, NTT

```
using ll = long long;

using cd = complex<double>;
const double PI = acos(-1.0);

/* ======================== FFT ======================== */

void fft(vector<cd>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang =
            2 * PI / len * (invert ? -1 : 1);
        cd wlen(cos(ang), sin(ang));
        for (int i = 0; i < n; i += len) {
            cd w(1);
            for (int j = 0; j < len / 2; j++) {
                cd u = a[i + j];
                cd v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
                w *= wlen;
            }
        }
    }
    if (invert) {
        for (cd& x : a)
            x /= n;
    }
}

vector<ll> multiply_fft(const vector<ll>& a, const vector<ll>& b) {
    if (a.empty() || b.empty())
        return {};
    int need = a.size() + b.size() - 1;
    int n = 1;
    while (n < need)
        n <<= 1;
    vector<cd> fa(a.begin(), a.end());
    vector<cd> fb(b.begin(), b.end());
    fa.resize(n);
    fb.resize(n);
    fft(fa, false);
    fft(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] *= fb[i];
    fft(fa, true);
    vector<ll> res(need);
    for (int i = 0; i < need; i++)
        res[i] = llround(fa[i].real());

    return res;
}

/* ======================== NTT ======================== */

const ll MOD = 998244353;
const ll ROOT = 3;

ll mod_pow(ll a, ll e) {
    ll r = 1;
    while (e) {
        if (e & 1)
            r = r * a % MOD;
        a = a * a % MOD;
        e >>= 1;
    }
    return r;
}

void ntt(vector<ll>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
            j ^= bit;
        j ^= bit;
        if (i < j)
            swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        ll wlen = mod_pow(
            ROOT,
            (MOD - 1) / len
        );
        if (invert)
            wlen = mod_pow(wlen, MOD - 2);
        for (int i = 0; i < n; i += len) {
            ll w = 1;
            for (int j = 0; j < len / 2; j++) {
                ll u = a[i + j];
                ll v = a[i + j + len / 2] * w % MOD;
                a[i + j] = u + v < MOD
                    ? u + v
                    : u + v - MOD;
```

```
                a[i + j + len / 2] = u - v >= 0
                    ? u - v
                    : u - v + MOD;
                w = w * wlen % MOD;
            }
        }
    }
    if (invert) {
        ll inv_n = mod_pow(n, MOD - 2);
        for (ll& x : a)
            x = x * inv_n % MOD;
    }
}

vector<ll> multiply_ntt(const vector<ll>& a, const vector<ll>& b) {
    if (a.empty() || b.empty())
        return {};
    int need = a.size() + b.size() - 1;
    int n = 1;
    while (n < need)
        n <<= 1;
    vector<ll> fa(a.begin(), a.end());
    vector<ll> fb(b.begin(), b.end());
    fa.resize(n);
    fb.resize(n);
    ntt(fa, false);
    ntt(fb, false);
    for (int i = 0; i < n; i++)
        fa[i] = fa[i] * fb[i] % MOD;
    ntt(fa, true);
    fa.resize(need);
    return fa;
}
```

Chinese Remainder Theorem

O(N)

modular arithmetic, number theory

```
/*
@title: Chinese Remainder Theorem
@category: Math
@complexity: O(N)
@tags: modular arithmetic, number theory
*/

struct Congruence {
    long long a, m;
};

long long mod_inv(long long a, long long m) {
    long long x, y;
    extgcd(a, m, x, y);
    return ((x % m) + m) % m;
}

long long chinese_remainder_theorem(vector<Congruence> const&
     congruences) {
    long long M = 1;
    for (auto const& congruence : congruences) {
        M *= congruence.m;
    }

    long long solution = 0;
    for (auto const& congruence : congruences) {
        long long a_i = congruence.a;
        long long M_i = M / congruence.m;
        long long N_i = mod_inv(M_i, congruence.m);
        solution = (solution + a_i * M_i % M * N_i) % M;
    }
    return solution;
}
```

Gaussian Elimination

O(N^3)

linear algebra, system of equations

```
/*
@title: Gaussian Elimination
@category: Math
@complexity: O(N^3)
@tags: linear algebra, system of equations
*/

const double EPS = 1e-9;
const int INF = 2; // it doesn't actually have to be infinity or a
     big number

int gauss (vector < vector<double> > a, vector<double> & ans) {
    int n = (int) a.size();
    int m = (int) a[0].size() - 1;

    vector<int> where (m, -1);
    for (int col=0, row=0; col<m && row<n; ++col) {
        int sel = row;
        for (int i=row; i<n; ++i)
            if (abs (a[i][col]) > abs (a[sel][col]))
                sel = i;
        if (abs (a[sel][col]) < EPS)
            continue;
        for (int i=col; i<=m; ++i)
            swap (a[sel][i], a[row][i]);
        where[col] = row;

        for (int i=0; i<n; ++i)
            if (i != row) {
                double c = a[i][col] / a[row][col];
                for (int j=col; j<=m; ++j)
                    a[i][j] -= a[row][j] * c;
            }
        ++row;
    }

    ans.assign (m, 0);
    for (int i=0; i<m; ++i)
        if (where[i] != -1)
            ans[i] = a[where[i]][m] / a[where[i]][i];
    for (int i=0; i<n; ++i) {
        double sum = 0;
        for (int j=0; j<m; ++j)
            sum += ans[j] * a[i][j];
        if (abs (sum - a[i][m]) > EPS)
            return 0;
    }

    for (int i=0; i<m; ++i)
        if (where[i] == -1)
            return INF;
    return 1;
}
```

XOR Basis

O(N log A)

linear algebra, system of equations

```
/*
@title: XOR Basis
@category: Math
@complexity: O(N log A)
@tags: linear algebra, system of equations
*/

#include <bits/stdc++.h>
using namespace std;

typedef long long ll;

ll rankGauss(vector<ll> x, ll y){
  vector<ll> basis(62,0);
  int r=0;
  int n=x.size();
  for(int i=0;i<n;i++){
    for(int bit=60;bit>=0;bit--){
      if((x[i]>>bit)&1){
        if(!basis[bit]){
          basis[bit]=x[i];
          r++;
          break;
        }
        x[i]^=basis[bit];
      }
    }
  }

  ll ty=y;
  for(int bit=60;bit>=0;bit--){
    if((ty>>bit)&1){
      if(basis[bit]){
        ty^=basis[bit];
      }
    }
  }
  if(ty!=0) return -1;

  // The number of subset that forms y is 2^(n-r) where r is the
     rank of the basis and n is the number of elements in the
     original set.

  return r;
}
```

06 Geometry

Faces of a Planar Graph

O(m log m)

planar graph, faces, embedding

```
/*
    Input:
        - Connected planar graph
        - Straight-line embedding
        - vertices[i] = coordinates of vertex i
        - adj[i] = neighbors of vertex i

    Output:
        faces[0] = outer face
        faces[i] = inner faces

    Inner faces are CCW.
    Outer face is CW.

    Vertices are 0-indexed.

    For a planar graph:
        m = O(n)
    so complexity is also O(n log n).

    If adjacency lists are already sorted by angle,
    the traversal itself can be made O(n).
*/

struct Point {
    long long x, y;

    Point(long long x = 0, long long y = 0)
        : x(x), y(y) {}

    Point operator-(const Point& p) const {
```

```
        return Point(x - p.x, y - p.y);
    }

    long long cross(const Point& p) const {
        return x * p.y - y * p.x;
    }

    int half() const {
        return y < 0 || (y == 0 && x < 0);
    }
};

vector<vector<int>> planar_faces(
    vector<Point> p,
    vector<vector<int>> adj
) {
    int n = p.size();

    // Sort neighbors counter-clockwise by polar angle.
    vector<vector<char>> used(n);

    for (int i = 0; i < n; i++) {
        used[i].assign(adj[i].size(), false);

        auto cmp = [&](int l, int r) {
            Point a = p[l] - p[i];
            Point b = p[r] - p[i];

            if (a.half() != b.half())
                return a.half() < b.half();

            return a.cross(b) > 0;
        };

        sort(adj[i].begin(), adj[i].end(), cmp);
    }

    vector<vector<int>> faces;

    // Every directed edge belongs to exactly one face.
    for (int s = 0; s < n; s++) {
        for (int e0 = 0; e0 < (int)adj[s].size(); e0++) {
            if (used[s][e0])
                continue;

            vector<int> face;

            int v = s;
            int e = e0;

            while (!used[v][e]) {
                used[v][e] = true;
                face.push_back(v);

                int u = adj[v][e];

                // Find v in adj[u].
                auto cmp = [&](int l, int r) {
                    Point a = p[l] - p[u];
                    Point b = p[r] - p[u];

                    if (a.half() != b.half())
                        return a.half() < b.half();

                    return a.cross(b) > 0;
                };

                int pos = lower_bound(
                    adj[u].begin(),
                    adj[u].end(),
                    v,
                    cmp
                ) - adj[u].begin();

                // Take the next edge clockwise.
                e = (pos + 1) % adj[u].size();
                v = u;
            }

            reverse(face.begin(), face.end());

            // Positive signed area -> inner face.
            // Non-positive -> outer face.
            Point p0 = p[face[0]];
            __int128 area = 0;

            for (int i = 0; i < (int)face.size(); i++) {
                Point a = p[face[i]];
                Point b = p[face[(i + 1) % face.size()]];

                area += (a - p0).cross(b - a);
            }

            if (area <= 0)
                faces.insert(faces.begin(), face);
            else
                faces.push_back(face);
        }
    }

    return faces;
}
```

05 Flows

Hungarian Algorithm

O(n^2 m), O(n^3) if n = m

assignment problem, minimum-cost matching

```
/*
    Minimum-cost assignment.
    A[i][j] = cost of assigning row i to column j.
    n <= m
    Returns:
        ans[i] = column assigned to row i
    Also returns the minimum total cost.
    1-indexed matrix A[1..n][1..m].
    For maximum cost:
        negate all A[i][j].
*/

pair<long long, vector<int>> hungarian(
    const vector<vector<long long>>& A
) {
    int n = (int)A.size() - 1;
    int m = (int)A[0].size() - 1;

    const long long INF = (1LL << 60);

    vector<long long> u(n + 1), v(m + 1);
    vector<int> p(m + 1), way(m + 1);

    for (int i = 1; i <= n; i++) {
        p[0] = i;
        int j0 = 0;
        vector<long long> minv(m + 1, INF);
        vector<char> used(m + 1, false);
        do {
            used[j0] = true;
            int i0 = p[j0];
            long long delta = INF;
            int j1 = 0;

            for (int j = 1; j <= m; j++) {
                if (used[j]) continue;

                long long cur = A[i0][j] - u[i0] - v[j];

                if (cur < minv[j]) {
                    minv[j] = cur;
                    way[j] = j0;
                }

                if (minv[j] < delta) {
                    delta = minv[j];
                    j1 = j;
                }
            }

            for (int j = 0; j <= m; j++) {
                if (used[j]) {
                    u[p[j]] += delta;
                    v[j] -= delta;
                } else {
                    minv[j] -= delta;
                }
            }
            j0 = j1;
        } while (p[j0] != 0);

        // Restore the augmenting path.
        do {
            int j1 = way[j0];
            p[j0] = p[j1];
            j0 = j1;

        } while (j0);
    }

    // p[j] = row assigned to column j.
    // Convert to ans[i] = column assigned to row i.
    vector<int> ans(n + 1);

    for (int j = 1; j <= m; j++)
        ans[p[j]] = j;

    return {-v[0], ans};
}
```

Dinic's Algorithm

O(V^2 * E), O(E * sqrt(V)) for bipartite graphs

max flow, residual graph

```
/*
@title: Dinic's Algorithm
@category: Graphs
@complexity: O(V^2 * E), O(E * sqrt(V)) for bipartite graphs
@tags: max flow, residual graph
*/

struct FlowEdge {
    int v, u;
    long long cap, flow = 0;
    FlowEdge(int v, int u, long long cap) : v(v), u(u), cap(cap) {}
};

struct Dinic {
    const long long flow_inf = 1e18;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;
```

```
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }

    void add_edge(int v, int u, long long cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, 0);
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }

    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }

    long long dfs(int v, long long pushed) {
        if (pushed == 0)
            return 0;
        if (v == t)
            return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid++) {
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u])
                continue;
            long long tr = dfs(u, min(pushed, edges[id].cap -
     edges[id].flow));
            if (tr == 0)
                continue;
            edges[id].flow += tr;
            edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }

    long long flow() {
        long long f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (long long pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
};
```

01 Strings

Aho-Corasick

O(N+M)

string matching, trie, aho-corasick

```
/*
@title: Aho-Corasick
@category: Strings
@complexity: O(N+M)
@tags: string matching, trie, aho-corasick
*/

#include <bits/stdc++.h>
using namespace std;

/*
Para frecuencias
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
  int nodes;
  vector<int> suf;
  vector<int> freq;
  vector<int> by_level;
  vector<bool> terminal;
  vector<array<int, E>> go;

  AhoCorasick() {
    add_node();
    nodes = 1;
  }

  void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    freq.emplace_back();
  }

  void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (go[pos][c] == 0) {
        go[pos][c] = nodes++;
        add_node();
      }
      pos = go[pos][c];
    }
    terminal[pos] = true;
  }

  void build() {
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
      int u = Q.front(); Q.pop();
      by_level.emplace_back(u);
      for (int c = 0; c < E; ++c) {
        if (go[u][c]) {
          int v = go[u][c];
          Q.emplace(v);
          suf[v] = u == 0 ? 0 : go[suf[u]][c];
        }
        else {
          go[u][c] = u == 0 ? 0 : go[suf[u]][c];
        }
      }
    }
  }
};*/
```

```
// Para existencia
template<int E = 26> // E = Alfabeto
struct AhoCorasick {
  int nodes;
  vector<int> suf;
  vector<int> super;
  vector<bool> terminal;
  vector<array<int, E>> go;

  AhoCorasick() {
    add_node();
    nodes = 1;
  }

  void add_node() {
    terminal.emplace_back();
    suf.emplace_back();
    go.emplace_back();
    super.emplace_back();
  }

  void insert(string &s) {
    int pos = 0;
    for (char ch : s) {
      int c = ch - 'a';
      if (go[pos][c] == 0) {
        go[pos][c] = nodes++;
        add_node();
      }
      pos = go[pos][c];
    }
    terminal[pos] = true;
  }

  void build() {
    queue<int> Q;
    Q.emplace(0);
    while (not Q.empty()) {
      int u = Q.front(); Q.pop();
      super[u] = (suf[u] == 0 or terminal[suf[u]] ? suf[u] : super
     [suf[u]]);
      for (int c = 0; c < E; ++c) {
        if (go[u][c]) {
          int v = go[u][c];
          Q.emplace(v);
          suf[v] = u == 0 ? 0 : go[suf[u]][c];
        }
        else {
          go[u][c] = u == 0 ? 0 : go[suf[u]][c];
        }
      }
    }
  }
};

template<int E = 26>
void mark(int u, AhoCorasick<E> &AC) {
  if (u == 0) return;
  mark(AC.super[u], AC);
  if (AC.terminal[u]) {
    cout << "El nodo " << u << " ocurre como subcadena" << endl;
    AC.terminal[u] = false;
  }
  AC.super[u] = 0;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  AhoCorasick<26> Solver;
  vector<string> S = {"arco", "co", "barco", "oro", "toro"};
```

```
  for (string s : S) {
    Solver.insert(s);
  }
  Solver.build();
  const string T = "coroparcotoro";
  int p = 0;
  for (char ch : T) {
    int c = ch - 'a';
    p = Solver.go[p][c];
    mark(p, Solver);
  }
  /*
  for (int i = (int)Solver.by_level.size() - 1; i > 0; --i) {
    int x = Solver.by_level[i];
    Solver.freq[Solver.suf[x]] += Solver.freq[x];
  }
  for (int i = 0; i < Solver.nodes; ++i) {
    cout << "El nodo " << i << " ocurre " << Solver.freq[i] << "
     veces" << endl;
  }*/
  return 0;
}
```

Manacher's Algorithm

O(N)

palindrome, string manipulation

```
/*
@title: Manacher's Algorithm
@category: Strings
@complexity: O(N)
@tags: palindrome, string manipulation
*/

vector<int> manacher_odd(string s) {
    int n = s.size();
    s = "$" + s + "^";
    vector<int> p(n + 2);
    int l = 0, r = 1;
    for(int i = 1; i <= n; i++) {
        if(i <= r) {
            p[i] = min(r - i, p[l + (r - i)]);
        }
        while(s[i - p[i]] == s[i + p[i]]) {
            p[i]++;
        }
        if(i + p[i] > r) {
            l = i - p[i], r = i + p[i];
        }
    }
    return vector<int>(begin(p) + 1, end(p) - 1);
}

vector<int> manacher(string s) {
    string t;
    for(auto c: s) {
        t += string("#") + c;
    }
    auto res = manacher_odd(t + "#");
    return vector<int>(begin(res) + 1, end(res) - 1);
}
```

Suffix Array

O(N log N)

suffix array, string matching

```
/*
@title: Suffix Array
@category: Strings
@complexity: O(N log N)
@tags: suffix array, string matching
*/

#include <bits/stdc++.h>
using namespace std;

vector<int> suffix_array(string s) {
  s += char(0);
  const int n = s.size();
  vector<int> a(n);
  iota(a.begin(), a.end(), 0);
  sort(a.begin(), a.end(), [&] (int i, int j) {
    return s[i] < s[j];
  });
  vector<int> mapping(n);
  mapping[a[0]] = 0;
  for(int i = 1; i < n; i++) mapping[a[i]] = mapping[a[i - 1]] + (
     s[a[i - 1]] != s[a[i]]);
  int len = 1;
  vector<int> sbs(n);
  vector<int> head(n);
  vector<int> new_mapping(n);
  while(len < n) {
    for(int i = 0; i < n; i++) sbs[i] = (a[i] + n - len) % n;
    for(int i = n - 1; i >= 0; i--) head[mapping[a[i]]] = i;
    for(int i = 0; i < n; i++) {
      int x = sbs[i];
      a[head[mapping[x]]++] = x;
    }
    new_mapping[a[0]] = 0;
    for(int i = 1; i < n; i++) {
      if(mapping[a[i - 1]] != mapping[a[i]]) {
        new_mapping[a[i]] = new_mapping[a[i - 1]] + 1;
      }
      else {
        int pre = mapping[(a[i - 1] + len) % n];
        int cur = mapping[(a[i] + len) % n];
        new_mapping[a[i]] = new_mapping[a[i - 1]] + (pre != cur);
      }
    }
    len <<= 1;
    swap(mapping, new_mapping);
  }
  return vector<int>(a.begin() + 1, a.end()); // Ignores a[0] since
      it's sentinel character.
}

vector<int> lcp_array(vector<int> &a, string s) {
  const int n = s.size();
  vector<int> rank(n);
  for(int i = 0; i < n; i++) rank[a[i]] = i;
  int k = 0;
  vector<int> lcp(n);
  for(int i = 0; i < n; i++) {
    if(rank[i] == n - 1) {
      k = 0;
      continue;
    }
    int j = a[rank[i] + 1];
    while(i + k < n and j + k < n and s[i + k] == s[j + k]) k++;
    lcp[rank[i]] = k;
    if(k) k--;
  }
  lcp.pop_back();
  return lcp;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  vector<int> a = suffix_array("abcbcba");
  vector<int> lcp = lcp_array(a, "abcbcba");
  for(auto x : a) cout << x << " \n"[x == a.back()];
  for(auto x : lcp) cout << x << " \n"[x == lcp.back()];
  return 0;
}
```

03 Trees

Heavy Light Decomposition

O(log^2 N)

HLD, path queries, segment tree

```
/*
@title: Heavy Light Decomposition
@category: Trees
@complexity: O(log^2 N)
@tags: HLD, path queries, segment tree
*/

#include <bits/stdc++.h>
using namespace std;

const int N = 5 * 100000 + 1;

int n;
int q;
int h[N];
int sz[N];
int par[N];
int nxt[N];
vector<int> G[N];

void DFS(int u) {
  sz[u] = 1;
  for (int i = 0; i < G[u].size(); ++i) {
    int v = G[u][i];
    DFS(v);
    sz[u] += sz[v];
    if (sz[v] > sz[G[u][0]]) {
      swap(G[u][i], G[u][0]);
    }
  }
}

void HLD(int u) {
  for (int v : G[u]) {
    h[v] = h[u] + 1;
    par[v] = u;
    nxt[v] = (v == G[u][0] ? nxt[u] : v);
    HLD(v);
  }
}

int lca(int u, int v) {
  while (nxt[u] != nxt[v]) {
    if (h[nxt[u]] > h[nxt[v]]) {
      u = par[nxt[u]];
    }
    else {
      v = par[nxt[v]];
    }
  }
  return h[u] < h[v] ? u : v;
}

int main() {
  cin.tie(0) -> sync_with_stdio(false);
  cin >> n >> q;
  for (int i = 1; i < n; ++i) {
    int p;
    cin >> p;
    G[p].emplace_back(i);
  }
  DFS(0);
  nxt[0] = 0;
  HLD(0);
}
```

```
while (q--) {
  int u, v;
  cin >> u >> v;
  cout << lca(u, v) << '\n';
}
return 0;
}
```